

OpenBalancer

Aggregating LLM Inference Capacity Across Providers

Devvjiit Bhuyan

June 5, 2026

Abstract

Large Language Model (LLM) applications increasingly depend on third-party inference providers such as OpenRouter, Groq, Hugging Face Inference, GitHub Models, Together AI, Fireworks AI, and others. While these providers simplify access to state-of-the-art models, they introduce common operational challenges including rate limits, outages, inconsistent latency, and vendor lock-in.

OpenBalancer is an OpenAI-compatible inference gateway designed to aggregate capacity across multiple providers while presenting a unified API surface to downstream applications.

Instead of relying on a single provider, OpenBalancer dynamically routes requests according to configurable policies including availability, latency, cost, quota utilization, and failover requirements.

The result is increased throughput, improved reliability, and reduced dependence on any individual inference provider.

1 Introduction

The rapid growth of LLM-powered applications has led to an ecosystem of inference providers offering access to open and proprietary models.

While these providers simplify deployment, developers often face operational constraints that limit application scalability and reliability.

OpenBalancer addresses these challenges by acting as an intelligent routing layer between applications and inference providers.

Applications communicate through a standard OpenAI-compatible interface while OpenBalancer determines the optimal backend provider for each request.

2 The Problem

Modern AI systems frequently depend on a single inference provider.

This architecture introduces several risks.

2.1 Rate Limits

Most providers enforce request-per-minute (RPM) and token-per-minute (TPM) restrictions.

Applications may operate correctly during development but experience degradation under increased demand.

2.2 Provider Outages

Temporary outages, service degradation, and networking failures can immediately impact application availability.

2.3 Vendor Lock-In

Applications tightly coupled to a single provider become difficult to migrate, optimize, or diversify.

2.4 Capacity Fragmentation

Developers often possess access to multiple inference providers.

However, capacity remains fragmented because applications typically utilize only one provider at a time.

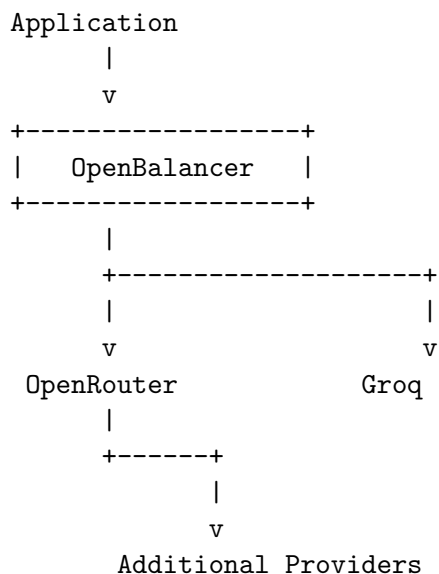
Unused capacity across providers therefore remains inaccessible.

3 OpenBalancer Architecture

OpenBalancer functions as an intelligent control plane for LLM inference.

Applications communicate with OpenBalancer through an OpenAI-compatible API.

OpenBalancer then routes requests to the most appropriate provider according to configured routing policies.



The application remains unaware of the selected backend provider.

4 Core Features

4.1 OpenAI API Compatibility

Existing OpenAI SDKs, LiteLLM clients, and direct HTTP integrations can connect to OpenBalancer by modifying only the base URL.

4.2 Intelligent Routing

OpenBalancer supports multiple routing strategies including:

- Priority routing
- Round-robin routing
- Least-loaded routing

- Cost-aware routing
- Latency-aware routing
- Failover routing

4.3 Automatic Failover

Provider failures automatically trigger fallback to alternate providers.
Applications continue operating without requiring modification.

4.4 Multi-Provider Capacity Aggregation

OpenBalancer combines the available throughput of multiple providers into a single logical endpoint.

4.5 Unified Observability

Provider usage, latency, request volume, and failure statistics can be monitored centrally.

5 Experimental Results

A benchmark was conducted using multiple free-tier providers.

5.1 Single Provider

A standalone provider typically offered approximately:

- ~30 RPM practical throughput
- before significant throttling occurred.

5.2 OpenBalancer

OpenBalancer distributed requests across:

- Groq
- Hugging Face Inference
- OpenRouter

Observed request distribution:

Provider	Requests
Groq	31
Hugging Face	14
OpenRouter	5

Table 1: Observed provider utilization during benchmark execution

Benchmark results:

- 50 successful requests

- 0 failures
- Approximately 12.7 requests per second

These results demonstrate successful aggregation of capacity across multiple independent providers.

6 Use Cases

6.1 Personal AI Projects

Increase throughput without purchasing dedicated inference infrastructure.

6.2 Retrieval-Augmented Generation

Maintain availability despite provider throttling and outages.

6.3 Multi-Agent Systems

Distribute inference requests across multiple providers to maximize throughput.

6.4 Enterprise Applications

Reduce dependence on a single vendor while maintaining a unified API layer.

6.5 Research Workloads

Utilize heterogeneous inference providers without modifying application code.

7 Future Directions

The OpenBalancer architecture enables several future capabilities:

- Dynamic provider scoring
- Adaptive latency-based routing
- Cost optimization
- Regional routing
- Multi-model balancing
- Local inference integration
- SLA-aware scheduling
- Provider benchmarking and ranking

8 Conclusion

OpenBalancer transforms multiple independent inference providers into a unified and resilient inference layer.

By aggregating capacity, reducing vendor lock-in, and enabling automatic failover, OpenBalancer allows developers to build more reliable AI systems without modifying application logic.

As the LLM ecosystem continues to fragment across providers, models, and pricing structures, intelligent routing will become increasingly important. OpenBalancer serves as the control plane that enables applications to leverage this diversity efficiently.